

AARHUS UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

MACHINE LEARNING

GROUP 30

Handin 1

Author

Søren M. DAMSGAARD

Alexander ERIKSEN

Thor BEHRMANN

Student number

202309814

202408929

202410271

October 9, 2026



Goofy ah Machines

PART 1: Logistic Regression

Theory

Question 1: Running Time of Mini-Batch Gradient Descent

Let n be the number of training samples, d the sample dimensionality, epochs the number of training epochs, and $B = \text{mini_batch_size}$ ($1 \leq B \leq n$). Multiplying an $(a \times b)$ matrix by a $(b \times c)$ matrix requires $\mathcal{O}(abc)$ time.

Cost computation: Given batch $X_{\text{batch}} \in \mathbb{R}^{B \times d}$, labels $y_{\text{batch}} \in \{-1, 1\}^B$, and weights $w \in \mathbb{R}^d$, computing $X_{\text{batch}}w$ takes $\mathcal{O}(B \cdot d)$ time. The element-wise product $y \odot (X_{\text{batch}}w)$ takes $\mathcal{O}(B)$. Computing $\ln(1 + e^{-y_i w^T x_i})$ via `np.logaddexp(0, -exponent)` across B elements and averaging takes $\mathcal{O}(B)$. Thus, cost computation on a batch of size B takes $\mathcal{O}(B \cdot d)$ time (and $\mathcal{O}(n \cdot d)$ on the full dataset of n samples).

Gradient computation: Evaluating sigmoid values $\sigma(-y_i w^T x_i)$ and scaling by $-y_i$ takes $\mathcal{O}(B)$ time. Multiplying X_{batch}^T (shape $d \times B$) by the length- B vector takes $\mathcal{O}(d \cdot B)$ operations. Scaling by $1/B$ takes $\mathcal{O}(d)$. Thus, gradient computation on a batch of size B takes $\mathcal{O}(B \cdot d)$ time.

Total running time of Mini-Batch Gradient Descent (fit): In each epoch, shuffling sample indices takes $\mathcal{O}(n)$ time. The dataset is partitioned into $\lceil n/B \rceil$ mini-batches; for each batch, computing the gradient takes $\mathcal{O}(B \cdot d)$ and updating weights $w \leftarrow w - \eta \nabla$ takes $\mathcal{O}(d)$. Summing across all batches yields $\frac{n}{B} \cdot \mathcal{O}(B \cdot d + d) = \mathcal{O}(n \cdot d)$. Evaluating the full in-sample cost at epoch end takes $\mathcal{O}(n \cdot d)$. Therefore, each epoch takes $\mathcal{O}(n \cdot d)$ time. Over all epochs, the total running time is:

$$\mathcal{O}(\text{epochs} \cdot n \cdot d)$$

Asymptotically, the running time per epoch is independent of the batch size B .

Question 2: Sanity Check (Pixel Permutation)

The classifier will achieve the **exact same** performance as with the raw data. Logistic regression treats each pixel as an independent feature coordinate x_j with its own weight w_j , without encoding 2D spatial locality (unlike CNNs). Applying any fixed permutation π across all image pixels simply permutes feature coordinate indices ($x_{\pi(j)}$) and correspondingly permutes the learned weights ($w_{\pi(j)}$), leaving the linear inner products $w^T x = \sum_j w_j x_j$, loss function, gradient steps, and decision boundaries completely invariant.

Question 3: Linearly Separable Data

For linearly separable data, the magnitude of the weights diverges to infinity ($\|w\| \rightarrow \infty$), while their direction converges to the maximum-margin separating hyperplane. The negative log-likelihood cost is $E_{\text{in}}(w) = \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-y_i w^T x_i})$. Since the data is linearly separable, there exists some w^* such that $y_i(w^{*T} x_i) > 0$ for all i . Scaling w^* by $c > 0$ yields:

$$\lim_{c \rightarrow \infty} E_{\text{in}}(cw^*) = \lim_{c \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n \ln(1 + e^{-cy_i w^{*T} x_i}) = 0$$

Because the infimum of 0 is only approached asymptotically as $c \rightarrow \infty$, gradient descent continuously takes descent steps that increase $\|w\| \rightarrow \infty$ without ever converging to finite weights.

Code

Summary and Results

We trained the Logistic Regression classifier on the Danish CVR industry codes dataset using bag-of-words text features (epochs = 50, batch_size = 16, lr = 0.1):

- **In-sample Accuracy:** 97.54%
- **Test Accuracy:** 95.34% (> 95% target benchmark)

Plot comments: Cost decreases smoothly from ~ 0.30 to ~ 0.08 with stable convergence around epoch 40, verifying that $\eta = 0.1$ is well-suited.

Status: All methods pass unit tests and gradient checks.

Actual Code

```
1 def cost_grad(self, X, y, w):
2     n, s = X.shape[0], y * np.dot(X, w) # Margin:  $s_i = y_i \cdot (x_i \cdot w)$ 
3     cost = np.mean(np.logaddexp(0, -s)) # Average NLL (numerically stable)
4     grad = -np.dot(X.T, y * logistic(-s)) / n # Gradient of average NLL
5     return cost, grad
6
7 def fit(self, X, y, w=None, lr=0.1, batch_size=16, epochs=10):
8     if w is None: w = np.zeros(X.shape[1])
9     self.history = []
10    n = X.shape[0]
11    for epoch in range(epochs):
12        perm = np.random.permutation(n)
13        for i in range(0, n, batch_size):
14            b = perm[i : i + batch_size]
15            _, grad = self.cost_grad(X[b], y[b], w)
16            w -= lr * grad
17        epoch_cost, _ = self.cost_grad(X, y, w)
18        self.history.append(epoch_cost)
19    self.w = w
```

Listing 1: Logistic Regression cost_grad and fit methods.

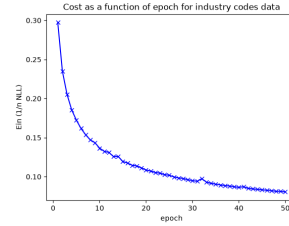


Figure 1: Cost (E_{in}) vs. epoch for industry codes.

PART 2: Softmax

Theory

Question: Running Time of Softmax Implementation

Assume classification with K classes, n training samples, d features, $B = \text{mini_batch_size}$ ($1 \leq B \leq n$), and epochs training iterations. Multiplying an $(a \times b)$ matrix by a $(b \times c)$ matrix takes $\mathcal{O}(abc)$ time.

Cost and Gradient in cost_grad: For mini-batch $X \in \mathbb{R}^{B \times d}$, targets $y \in \{0, \dots, K-1\}^B$, and weights $W \in \mathbb{R}^{d \times K}$: one-in- K encoding Y_k takes $\mathcal{O}(B \cdot K)$; logits XW multiplies $(B \times d)$ by $(d \times K)$ taking $\mathcal{O}(B \cdot d \cdot K)$ operations; row-wise softmax probabilities P (max-subtraction, exponentiation, sum, normalization) take $\mathcal{O}(B \cdot K)$; cost $-\frac{1}{B} \sum_{i,k} Y_{ik} \ln P_{ik}$ takes $\mathcal{O}(B \cdot K)$; gradient $\frac{1}{B} X^T (P - Y_k)$ multiplies $(d \times B)$ by $(B \times K)$ taking $\mathcal{O}(d \cdot B \cdot K)$ operations and $\mathcal{O}(d \cdot K)$ for scaling. Thus, `cost_grad` runs in $\mathcal{O}(B \cdot d \cdot K)$ time per mini-batch (and $\mathcal{O}(n \cdot d \cdot K)$ on the full dataset).

Total running time of Mini-Batch Gradient Descent (fit): In each epoch, shuffling takes $\mathcal{O}(n)$ time. Processing all $\lceil n/B \rceil$ mini-batches takes $\sum_{j=1}^{\lceil n/B \rceil} \mathcal{O}(B \cdot d \cdot K + d \cdot K) = \mathcal{O}(\frac{n}{B} \cdot B \cdot d \cdot K) = \mathcal{O}(n \cdot d \cdot K)$. Evaluating the full in-sample cost at epoch end takes $\mathcal{O}(n \cdot d \cdot K)$. Across all epochs, the total running time is:

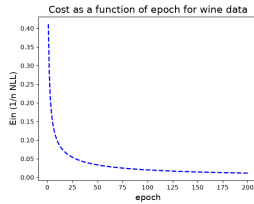
$$\mathcal{O}(\text{epochs} \cdot n \cdot d \cdot K)$$

Code

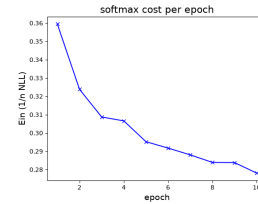
Summary and Results

We evaluated our Softmax Classifier on both the Wine and MNIST digits datasets:

- **Wine Dataset** (200 epochs, $B = 16$, $\eta = 0.1$): In-sample Accuracy: 100.0%, Test Accuracy: 97.78% ($> 90\%$ benchmark).
- **MNIST Digits** (10 epochs, $B = 32$, $\eta = 0.05$): In-sample Accuracy: 92.39%, Test Accuracy: 92.28%.



a Cost per epoch on Wine data.



b Cost per epoch on MNIST digits.

Figure 2: Training cost progression for Softmax regression.

Comments on the plots: Both cost curves show steep initial drops followed by steady flattening without oscillations, reflecting smooth convergence. Numerical stability (log-sum-exp trick) completely prevented overflow during all runs.

Actual Code

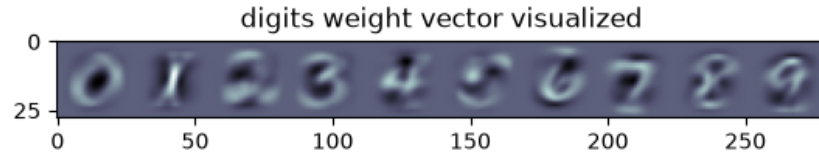


Figure 3: Visualization of learned weight templates for MNIST digits 0–9. Bright values correspond to positive weights matching expected stroke shapes; dark values indicate negative weights for background pixels.

```
1 def cost_grad(self, X, y, W):
2     Yk = one_in_k_encoding(y, self.num_classes)
3     n = X.shape[0]
4     P = softmax(np.dot(X, W))
5     cost = -np.sum(Yk * np.log(P)) / n # Average NLL cost
6     grad = np.dot(X.T, P - Yk) / n    # Gradient of average NLL
7     return cost, grad
8
9 def fit(self, X, Y, W=None, lr=0.01, epochs=10, batch_size=16):
10     if W is None: W = np.zeros((X.shape[1], self.num_classes))
11     self.history = []
12     n = X.shape[0]
13     for epoch in range(epochs):
14         perm = np.random.permutation(n)
15         for i in range(0, n, batch_size):
16             b = perm[i : i + batch_size]
17             _, grad = self.cost_grad(X[b], Y[b], W)
18             W -= lr * grad
19         epoch_cost, _ = self.cost_grad(X, Y, W)
20         self.history.append(epoch_cost)
21     self.W = W
```

Listing 2: Softmax cost_grad and fit methods.